

Database recovery and Atomicity:-

Database recovery and Atomicity:-

- ☐ Introduction
- ☐ recovery algorithms
- ☐ log base recovery
- ☐ shadow paging
- ☐ recovery with concurrent transaction
- ☐ Checkpoints or sync points or save points

Query Optimization:

- ☐ Overview Query Processing and Optimization
- ☐ Heuristics and Cost Estimates in Query Optimization.

Database recovery and Atomicity

The database system must take actions in advance to ensure that atomicity and durability properties of transactions, are subject to variety of failure: disk crash, power outage, software error, a fire in the machine room, even sabotage. In any failure, information may be lost.

Example: A transaction that transfers Rs.50 from account A to account B, with initial values of A and B being Rs.1000 and Rs.2000, respectively.

Schedule S ₁	
T _i	Explanation
Read (A,a);	A=1000;
a=a - 50;	950=1000-50;
Write (A,a);	A=950
Read (B, b);	B=2000
b= b + 50;	
Write (B, b);	

Transaction T_i fails after
Write (A, a) ←

Suppose that a system crash has been occurred during the execution of T_i, after write (A) has taken place, but before write (B) was executed, where a and b denote the buffer blocks on which A and B reside. Since the memory contents were lost, we do not know the outcome of the transaction; thus, Two possible recovery procedures:

- **Reexecute T_i:** This procedure will result in the value of A becoming \$900, rather than \$950. Thus, the system enters an inconsistent state.
- **Do not reexecute T_i:** The current system state has values of \$950 and \$2000 for A and B, respectively. Thus, the system enters an inconsistent state.

In either case, the database is gone in an inconsistent state and simple recovery scheme does not work. The difficulty is that we have modified the database without having guaranteed that the transaction will surely commit.

The process of restoring the database to consistent state from its inconsistent state that has been caused due to failure is known as database recovery.

DBMS ensure database reliability i.e capability of DBMS to recover various types of failure. It is the responsibility of the recovery manager to handle such failure and ensure 'atomicity' and 'durability'. It manages atomicity by undoing the uncommitted transactions. It also manage durability by maintain the committed transaction results even after system crashes.

In a database recovery, mainly two recovery techniques that can help a DBMS to recovering and maintaining the atomicity of a transaction. Those are as follows

1. Log Based Recovery.

2. Shadow Paging

Recovery Algorithms

Recovery algorithms to ensure database consistency and transaction atomicity, durability regardless of failures. Recovery algorithms have two parts:

1. Actions taken during normal transaction processing is to guarantee that sufficient information exists to recover from failures,
2. Actions taken after a failure to recover the database contents to a state ensure atomicity, consistency and durability.

In above example, transaction **T_i** transfers Rs.50 from account **A** to account **B**; goal is either to perform all database modifications made by **T_i** or none at all. **T_i** modifies **A** by subtracting Rs. 50 and modifies **B** by adding Rs. 50. A failure may occur after one of these modifications has been made, but before all of them are made.

In the recovery method, we use two operations:

1. **Undo (T_i):** Restores to old values of all data item updated by transaction **T_i**.
2. **Redo (T_i):** Sets new values of all data item updated by transaction **T_i**.

We need to undo a transaction **T** only when log contains the record <T start> only when log contains the record <start> and <T commit> both.

There are two main techniques for recovery from non-catastrophic transaction failures:

1. Deferred update (NO UNDO/REDO) algorithm.
2. Immediate update (UNDO/REDO) algorithm.

1. Deferred update:

- These techniques do not physically update database on disk until after a transaction reaches its commit point. Before reaching commit, all transaction updates are recorded in the buffers.
- During commit, the updates are first recorded persistently in the log and then written in the database.
- If a transaction fails before reaching its commit point, it cannot change database, so UNDO is not needed.
- It may be necessary REDO the effect of operations of a committed transaction from the log, because their effect may not yet have been recorded in the database. **Hence, deferred update also called as the NO-UNDO/REDO algorithm.**

2. Immediate update:

- The database may be updated by some operations of a transaction before transaction reaches its commit point.
- These operations are recorded in the log on disk.
- If a transaction fails after recording some changes in the database but before reaching its commit point, the effect of its operations on the database must be undone i.e., the transaction must be rolled back.
- Both undo and redo may be required during recovery. This technique is known as UNDO/REDO algorithm.

Log base recovery

Log is a sequence of records, which maintains the records of actions performed by a transaction. A log record is useful to recovery of system and disk failures. Log-based recovery works as follows –

- A log file is kept on a stable storage.
- When a transaction enters in the system and starts execution, it writes a log.

The log file contains the following information:

- **Transaction identifier** is a unique identifier of transaction that performed write operation.
- **Type of log record** (transaction start, insert, update, delete, abort, commit)
- **Identifier of data item** (table, row, column) affected by the database action (DML operations)
- **Before-image of data item:** The old value of the data item (Before the write operation).
- **After-image of data item:** The new value of the data item (After the write operation).
- **Log management information** such as pointer to previous and next log records of transaction.

Types of log record

- 1) <Start> Log Record
- 2) <Update> Log Record

3) <Commit> Log Record**4) <Abort> Log Record**

Log records as:

- **<Ti start>**: Transaction Ti has started.
- **<Ti, Xj, V1, V2>**: Transaction Ti has performed a write on **data item Xj**. Xj had value *V1* (Old value) before write and value *V2* (new value) after write.
- **<Ti commit>**: Transaction Ti has committed.
- **<Ti abort>**: Transaction Ti has aborted.

Consider the data Item A and B with initial value 1000. (A=B=1000)

Transaction T	Log Record
Read(A)	$\langle T_i \text{ Start} \rangle$
A=A-50	$\langle T_i, A, 1000, 950 \rangle$
Write(A)	$\langle T_i, B, 1000, 1050 \rangle$
B=B+50	$\langle T_i \text{ commit} \rangle$.
Write(B)	
Commit	

1. Whenever a transaction performs a write, it is important that the log record to be created before the database is modified.
2. Once a log record exists, we can output the modification into database. Also, we have ability to undo the modification.

There are two techniques for log-based recovery as follows:

1. Deferred Database Modification**2. Immediate Database Modification****1. Deferred Database Modification**

- Updates (changes) to the database are deferred (or postponed) until the transaction commits.
- During the execution of transaction, updates are recorded only in the transaction log and in buffers. After the transaction commits, these updates are recorded in the database.

When failure occurs

- If the transaction fails or crashes after commit operation. Recovery manager will check the log file, and find both $\langle T, \text{Start} \rangle$ and $\langle T, \text{Commit} \rangle$. A REDO (T) operation will be performed and Updated values of data items will be set in Database.
- If Transaction failure before executing commit operation. Recovery manager will check the Log file and find $\langle T, \text{Start} \rangle$ entry in the Log file but not find the $\langle T, \text{Commit} \rangle$ entry. So it ensure atomicity property, **Recovery Manager** will set the old value of data items. No need of undo operation.

Only the new value of data item is required by deferred modification technique because if the transaction fails before commit then log record is ignored because data item already contain old value due to deferred write operation.

If transaction commit, the new value present in the log record are copied into database item. Thus log record contains only new values of data item, not old values of data item. This approach save the stable storage space and reduce the complexity of log record.

Example: Consider the following two transactions, T1 and T2 given in figure, where T1 executes before T2. Consider that initial values for A, B and C are 500, 600 and 700 respectively.

T ₁	T ₂
Read (A, a); a=a - 100; Write (A, a); Read (B, b); b = b + 1 00; Write (B, b);	Read (C, c); c = c - 200; Write (C, c);

Shows transaction log for above two transactions at three different instances of time.

Time Instance (a)	Time Instance (b)	Time Instance (c)
<T1 start>	<T1 start>	<T1 start>
< T1, A, 500,400>	< T1, A,500, 400>	< T1, A,500, 400>
< T1, B, 600,700>	< T1, B,600,700>	< T1, B, 600,700>
	<T1 commit>	<T1 commit>
	<T2 start>	<T2 start>
	<T2 C,700,500>	<T2 C, 700,500>
		<T2 commit>

If failure occurs in case of

1. No any UNDO actions are required.
2. As Transaction T1 has already committed, it must be redone.
3. As Transactions T1 and T2 have already committed, they must be redone.

2. Immediate Database Modification:

- Updates (changes) to the database are applied immediately as they occur without waiting to reach to the commit point.
- these updates are recorded in the transaction log.
- It is possible here that updates of the uncommitted transaction are also written to the database. And, other transactions can access these updated values.

When failure occurs

- If the transaction fails or crashes after commit operation. Recovery manager will check log file, and find both <T, Start> and <T, Commit> in the Log file. It represents that Transaction T has been completed successfully before the system failed so **REDO (T)** operation will be performed and Updated values of dataitem will be set in database.
- If Transaction failure before executing commit operation. Recovery manager will check the Log file and find < T, Start> entry in the Log file but not find the <T, Commit> entry. So it ensure atomicity property **UNDO (T)** operation will be performed because Update Values are written in the Database immediately after the write operation. So Recovery manager will set old value to data item.

Example: consider the two transactions, T1 and T2, given in figure, where T1 executes before T2.

- Also consider that initial values for A, B and C are 500, 600 and 700 respectively.
- The following figure shows the transaction log for above two transactions at three different instances of time. Transaction log contains original values also along with new updated values for data items.
- If failure occurs in case of –
 - Undo the transaction T1 as it has not committed, and restore A and B to 500 and 600 respectively.
 - Undo the transaction T2, restore C to 700; and, Redo the Transaction T1 set A and B to 400 and 700 respectively.
 - Redo the Transaction T1 and Transaction T1; and, set A and B to 400 and 700 respectively, while set C to 500.

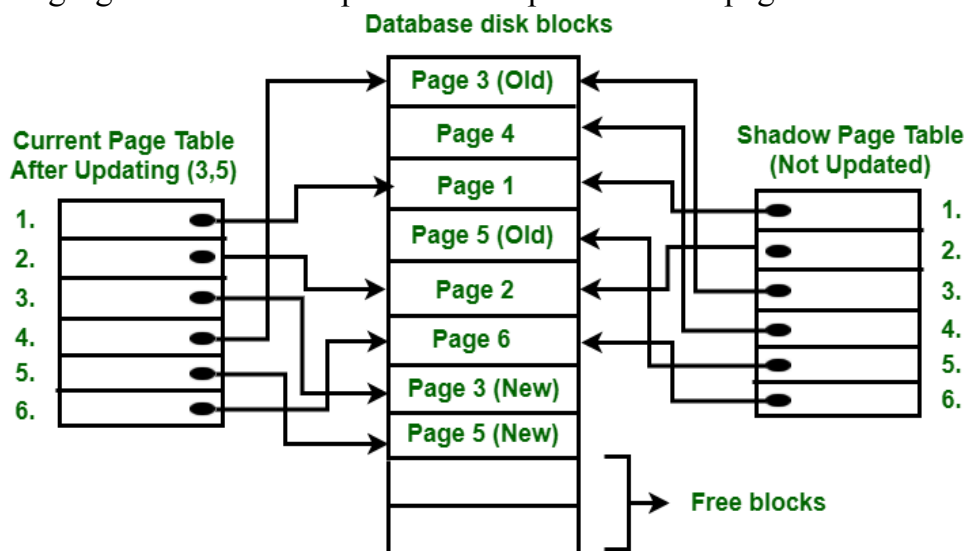
Time Instance (a)	Time Instance (b)	Time Instance (c)
<T1 start>	<T1 start>	<T1 start>
< T1, A, 500,400>	< T1, A, 500,400>	< T1, A, 500,400>
< T1, B, 600,700>	< T1, B, 600,700>	< T1, B, 600,700>
	<T1 commit>	<T1 commit>
	<T2 start>	<T2 start>
	<T2 C, 700,500>	<T2 C, 700,500>
		<T2 commit>

Shadow Paging Technique.

- Shadow paging is an alternative to transaction-log based recovery techniques.
- Shadow Paging is a recovery technique that provides atomicity and durability in database system.
- A database is divide into fixed size disk blocks called pages. These pages are mapped to physical storage using a table called page table.
- The page table is indexed by a page number of the database. The information about physical pages, in which database pages are stored, is kept in this page table.
- This technique is similar to paging technique used by Operating System to allocate memory.

Execution of Transaction

- During the execution of the transaction, two page tables are maintained.
 - Current Page Table:** Used to access data items during transaction execution.
 - Shadow Page Table:** Original page table, and will not modified during transaction execution.
- Before starting a transaction, both tables are same and point to same pages.
- The shadow page table is never changed, and used to restore database in the occurrence any failure. But current page table entries may change during execution of transaction, as it is used to record all updates made to the database.
- When the transaction completes, Current page table becomes same as shadow page table and transaction is committed. Buffers are transferred to physical database.
- Consider following figure. Two write operations are performed on page 3 and 5.



Before start of write operation on page 3, current page table points to old page 3. When write operation starts following steps are performed:

- Firstly, search start for available free block in disk blocks.
- After finding free block, it copies page 3 to free block which is represented by Page 3 (New).
- Now current page table points to Page 3 (New) on disk but shadow page table points to old page 3 because it is not modified.
- The changes are now circulated to Page 3 (New) which is pointed by current page table.

Failure Operation:

- If system failure before commit operation, it is sufficient to free modified database pages and remove current page table. Database recovered by reinstalling shadow page table.
- If system failure after last write operation then it does not affect propagation of changes made by

transaction. These changes are preserved and there is no need to perform redo operation.

Advantages:

- Require fewer disk accesses to perform operation.
- Recovery from crash is inexpensive and quite fast.
- There is no need of operations like- Undo and Redo.

Disadvantages:

- Copying the entire page table is very expensive.
- Data are scatters, or fragmented.
- After each transaction, free pages need to be collected by garbage collector.
- Difficult to extend this technique to allow concurrent transactions.
- The commit of single transaction requires multiple blocks which decreases execution speed.
- To allow this technique to multiple transactions concurrently it is difficult.

• Recovery with Concurrent Transaction

When more than one transaction is being executed in parallel, the logs are interleaved. At the time of recovery, it would become hard for the recovery system to backtrack all logs, and then start recovering. To ease this situation, most modern DBMS use the concept of 'checkpoints'. There are four ways of recovery from concurrent transactions. They are as follows:

1. Interaction with concurrency control:

Recovery depends upon concurrency control scheme. So, rollback failed transaction, we must undo the updates performed by the transaction.

2. Transaction rollback:

We rollback a failed transaction by using log. The system scans log backward, For every record found in the log; the system restores the data item.

3. Checkpoints:

We use checkpoints to reduce the number of log records that the system must scan when it recovers from a crash. In a concurrent transaction processing system, we require that the checkpoint log record be of the form—

<Checkpoint Q>

Where 'Q' is a list of transactions active at the time of the checkpoint.

A **fuzzy checkpoint** is a checkpoint where transactions are allowed to perform update even while buffer blocks are being written out. We shall be describing checkpoints in the next section again.

4. Restart recovery:

When the system recovers from a crash, it constructs two lists

- Undo-list: consists of transactions to be undone.
- Redo-list: consists of transaction to be redone.

System follows:

a) Initially, both lists are empty.

b) The system scans log backwards, examining each record, until it finds the first<checkpoint> record.

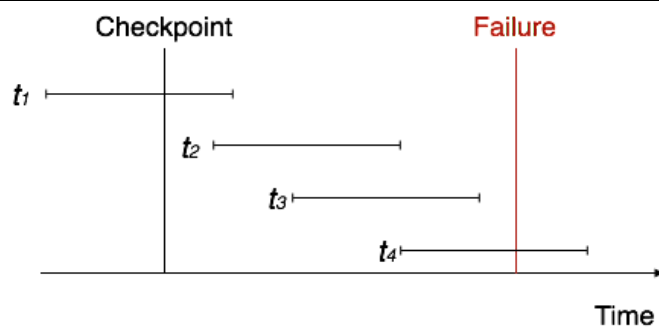
• Check Points/Sync points/Save Points

- Checkpoint (or sync point) is synchronization point between database and transaction log file.
- When the size of the transaction log file becomes too large to manage and handle. The DBMS checkpoint is a mechanism to remove all previous logs from the system and permanently stored in the storage disk. It look likes bookmark.
- The checkpoint is used to declare a point before which the DBMS was in the consistent state, and all transactions were committed.
- When it reaches to the checkpoint, then the transaction will be updated into the database, and till that point, the entire log file will be removed from the file. Then the log file is updated with the new step of transaction till next checkpoint and so on.

Recovery using Checkpoint

In the following manner, a recovery system recovers the database from this failure:

T1	T2	T3	T4
----	----	----	----



Start			
	Start		
	Commit		
		Start	
		Commit	
			Start
			Failure

Operations	List
$\langle T_n, \text{start} \rangle$ and $\langle T_n, \text{commit} \rangle$	Redo list
$\langle T_n, \text{commit} \rangle$	Redo list
$\langle T_n, \text{start} \rangle$	Undo list

- The recovery system reads log files from end to start i.e. reads log files from T4 to T1.
- Recovery system maintains and creates two lists: redo-list and an undo-list.
- If the transactions contain operations like $\langle T_n, \text{start} \rangle$ and $\langle T_n, \text{commit} \rangle$ together or $\langle T_n, \text{commit} \rangle$ alone then the transaction will be stored in the redo list.
- **Example:** Transactions T2 and T3 contain $\langle T_n, \text{start} \rangle$ and $\langle T_n, \text{commit} \rangle$ both the operations but Transaction T1 contains only $\langle T_n, \text{Start} \rangle$. Hence, transactions T2 and T3 are stored in the redo list. All transactions are removed and redone before saving their logs.
- If the transactions contain operations like $\langle T_n, \text{start} \rangle$ but not $\langle T_n, \text{commit} \rangle$, then the transaction will be stored in undo list(T1).
- **Example:** The transaction T4 contains $\langle T_n, \text{start} \rangle$ but not $\langle T_n, \text{commit} \rangle$ operation. Thus transaction T4 is stored in undo list. All the transactions are undone, and their logs are removed.

Advantages of Checkpoint

- Enhances the consistency of the database during the execution of multiple transactions concurrently.
- Help in getting our transactions restore in the case of accidental shutdown of the database.
- Writing all the dirty pages from log files or the buffer to physical disk.
- Quick data recovery process.
- Checkpoint records in the log file avoid irrelevant redo operations.
- The execution and the maintenance of the transaction log file become easier and simpler due to the continuous shifting of dirty pages from log files to permanent storage.

Drawbacks of using Check Pointing

- Searching operation is time consuming and Most of transactions might to be redone
- The system repeatedly executes checkpoints.
- The transactions are not granted to perform any modification while the checkpoints are in progress.
- The existence of a checkpoint allows the system to integrate its recovery process.
- Uses deferred or immediate modification techniques, system performs checkpoint regularly.

Query Optimization

The techniques used to split complex queries into multiple simple operations and methods of implementing these low-level operations. **Query optimization is the process of selecting an efficient execution plan for evaluating query with minimize cost.** A query optimizer decides the best methods for implementing each query.

Optimizer to calculate the cost of a query evaluation plan is:

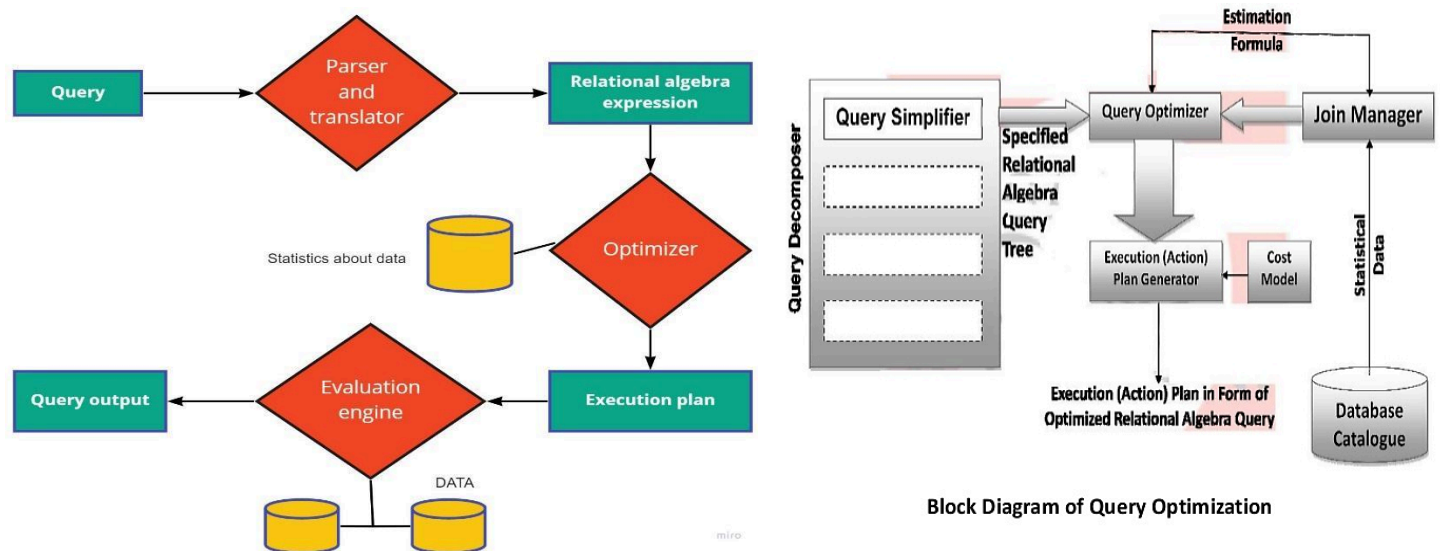
- CPU time
- Number of tuples to be scanned
- Disk access time
- number of operations

Query processing is group of phases associated with the extraction of data from the database. It conversion of queries written in a high-level language (SQL) into low-level Language which is understand by internal level of database.

Three different phases during the query processing in DBMS which are as follows:

User queries are submitted to DBMS as SQL queries.

- **Parsing and translation:** the given query is translated into its internal form and checks the syntax of the user's query, verifies that the relation names appearing in the query are names of the relations in the database. The system constructs a parse tree representation of the query, which is translate into a relational algebra expression.
- **Optimization:** A SQL query can be written in many different ways. An optimized query also depends on how the data is stored in the file organization. A Query can also have different corresponding relational algebra expressions. Query optimizer generates different execution plans to evaluate parsed query and select the best plan with least estimated cost. Catalog manager helps optimizer to choose best plan to execute query generating cost of each plan.
- **Execution plan:** A systematic step-by-step execution of basic operations for fetching data from database is called as query evaluation plan. Different evaluation plans for a particular query have different query costs. The cost may include the number of disk accesses, CPU time for execution of the query, time of communication in the case of distributed databases.



Block Diagram of Query Optimization

Techniques for Query Optimization

1. Cost based Optimization (Physical)

This is based on the cost of the query. The query can use different paths based on indexes,

constraints, sorting methods etc. This method mainly uses the statistics like record size, number of records, number of records per block, number of blocks, table size, whether whole table fits in a block, organization of tables, uniqueness of column values, size of columns etc.

Suppose, we have series of table joined in a query.

$$T1 \propto T2 \propto T3 \propto T4 \propto T5 \propto T6$$

For above query we can have any order of evaluation. We can start taking any two tables in any order and start evaluating the query. Ideally, we can have join combinations in $(2(n-1))! / (n-1)!$ ways. For example, suppose we have 5 tables involved in join, then we can have $8! / 4! = 1680$ combinations. But when query optimizer runs, it does not evaluate in all these ways always. It uses Dynamic Programming where it generates the costs for join orders of any combination of tables. It is calculated and generated only once. This least cost for all the table combination is then stored in the database and is used for future use. i.e.; say we have a set of tables, $T = \{ T1, T2, T3 \dots Tn \}$, then it generates least cost combination for all the tables and stores it.

Dynamic Programming

The least cost for the joins of any combination of table is generated here. These values are stored in the database and when those tables are used in the query, this combination is selected for evaluating the query.

While generating the cost, it follows below steps :

Suppose we have set of tables, $T = \{T_1, T_2, T_3 \dots T_n\}$, in a DB. It picks the first table, and computes cost for joining with rest of the tables in set T. It calculates cost for each of the tables and then chooses the best cost. It continues doing the same with rest of the tables in set T. It will generate $2n - 1$ cases and it selects the lowest cost and stores it. When a query uses those tables, it checks for the costs here and that combination is used to evaluate the query. This is called dynamic programming.

In this method, time required to find optimized query is in the order of $3n$, where n is the number of tables. Suppose we have 5 tables, then time required is $3 \times 5 = 15$, which is lesser than finding all the combination of tables and then deciding the best combination (1680). Also, the space required for computing and storing the cost is also less and is in the order of $2n$. In above example, it is $2 \times 5 = 10$.

Left Deep Trees

This is another method of determining the cost of the joins. Here, the tables and joins are represented in the form of trees. The joins always form the root of the tree and table is kept at the right side of the root. LHS of the root always point to the next join. Hence it gets deeper and deeper on LHS. Hence it is called as left deep tree.

Here instead of calculating the best join cost for set of tables, best join cost for joining with each table is calculated. In this method, time required to find optimized query is in the order of n^2 , where n is the number of tables. Suppose we have 5 tables, then time required is $5 \times 5 = 25$, which is lesser than dynamic programming. Also, the space required for computing storing the cost is also less and is in the order of $2n$. In above example, it is $2 \times 5 = 10$, same as dynamic programming.

Interesting Sort Orders

This method is an enhancement to dynamic programming. Here, while calculating the best join order costs, it also considers the sorted tables. It assumes, calculating the join orders on sorted tables would be efficient. i.e.; suppose we have unsorted tables $T_1, T_2, T_3 \dots T_n$ and we have join on these tables.

$$(T_1 \bowtie T_2) \bowtie T_3 \bowtie \dots \bowtie T_n$$

This method uses hash join or merge join method to calculate the cost. Hash Join will simply join the tables. We get sorted output in merge join method, but it is costlier than hash join. Even though merge join is costlier at this stage, when it moves to join with third table, the join will have less effort to sort the tables. This is because first table is the sorted result of first two tables. Hence it will reduce the total cost of the query.

But the number of tables involved in the join would be relatively less and this cost/space difference will be hardly noticeable.

All these cost based optimizations are expensive and are suitable for large number of data. There is another method of optimization called heuristic optimization, which is better compared to cost based optimization.

2. Heuristic Optimization (Logical)

This method is also known as rule based optimization. This is based on the equivalence rule on relational expressions; hence the number of combination of queries get reduces here. Hence the cost of the query too reduces.

This method creates relational tree for the given query based on the equivalence rules. These equivalence rules by providing an alternative way of writing and evaluating the query, gives the better path to evaluate the query. This rule need not be true in all cases. It needs to be examined after applying those rules. The most important set of rules followed in this method is listed below:

- Perform all the selection operation as early as possible in the query. This should be first and foremost set of actions on the tables in the query. By performing the selection operation, we can reduce the number of records involved in the query, rather than using the whole tables throughout the query.

Suppose we have a query to retrieve the students with age 18 and studying in class DESIGN_01. We can get all the student details from STUDENT table, and class details from CLASS table. We can write this query in two different ways.

```
SELECT std.* FROM STUDENT std, CLASS cls
WHERE std.CLASS_ID = cls.CLASS_ID
AND std.AGE = 18
AND cls.CLASS_NAME = 'DESIGN_01';
```

```
SELECT std.* FROM
(SELECT * FROM STUDENT WHERE AGE = 18) std,
(SELECT * FROM CLASS WHERE CLASS_NAME = 'DESIGN_01') cls
WHERE std.CLASS_ID = cls.CLASS_ID
```

Here both the queries will return same result. But when we observe them closely we can see that first query will join the two tables first and then applies the filters. That means, it traverses whole table to join, hence the number of records involved is more. But the second query, applies the filters on each table first. This reduces the number of records on each table (in class table, the number of records reduces to one in this case!). Then it joins these intermediary tables. Hence the cost in this case is comparatively less.

Instead of writing query the optimizer creates relational algebra and tree for above case.

- Perform all the projection as early as possible in the query. This is similar to selection but will reduce the number of columns in the query.

Suppose for example, we have to select only student name, address and class name of students with age 18 from STUDENT and CLASS tables.

Here again, both the queries look alike, results alike. But when we compare the number of records and attributes involved at each stage, second query uses less records and hence more efficient.

- Next step is to perform most restrictive joins and selection operations. When we say most restrictive joins and selection means, select those set of tables and views which will result in comparatively less number of records. Any query will have better performance when tables with few records are joined. Hence throughout heuristic method of optimization, the rules are formed to get less number of records at each stage, so that query performance is better. So is the case here too.

Suppose we have STUDENT, CLASS and TEACHER tables. Any student can attend only one class in an academic year and only one teacher takes a class. But a class can have more than 50 students.

Now we have to retrieve STUDENT_NAME, ADDRESS, AGE, CLASS_NAME and TEACHER_NAME of each student in a school.

```
[]STD_NAME, ADDRESS, AGE, CLASS_NAME, TEACHER_NAME ((STUDENT ∞ CLASS_ID CLASS) ∞ TECH_IDTEACHER)
```

Not So efficient

```
[]STD_NAME, ADDRESS, AGE, CLASS_NAME, TEACHER_NAME (STUDENT ∞ CLASS_ID (CLASS ∞ TECH_IDTEACHER))
```

Efficient

In the first query, it tries to select the records of students from each class. This will result in a very huge intermediary table. This table is then joined with another small table. Hence the traversing of number of records is also more. But in the second query, CLASS and TEACHER are joined first, which has one to one relation here. Hence the number of resulting record is STUDENT table give the final result. Hence this second method is more efficient.

- Sometimes we can combine above heuristic steps with cost based optimization technique to get better results.

All these methods need not be always true. It also depends on the table size, column size, type of selection, projection, join sort, constraints, indexes, statistics etc. Above optimization describes the best way of optimizing the queries.